

TAMPER EVIDENT LOGGING SYSTEM

Submitted by:
Jhanvi Patel

TABLE OF CONTENTS

1. Introduction
2. Task: Tamper-Evident Logging System
3. Implementation Details
4. Why Python Was Chosen
5. Results and Observations
6. Conclusion
7. References

1. Introduction

Cybersecurity is an important part of protecting systems, data, and applications from unauthorized access and harmful activities. As we rely more on digital platforms in everyday life, it becomes essential to ensure that data remains secure and that systems behave as expected.

In this assignment, a key cybersecurity concepts are implemented: a tamper-evident logging system. The tamper-evident logging system is designed to detect any unauthorized changes made to stored logs using cryptographic techniques.

This Task was developed using Python because of its simplicity and the availability of useful built-in libraries. This made it easier to demonstrate important security concepts such as hashing, input validation, and controlled execution in a clear and practical way.

Overall, this assignment helped in understanding how basic security mechanisms can be applied in real-world scenarios. It also highlights the importance of building systems that are not only functional but also secure and reliable.

2. Task: Tamper – Evident Logging System

2.1 Objective

The objective of this task is to design and implement a logging system that can detect any unauthorized changes made to stored logs. The system ensures data integrity by linking each log entry using cryptographic hashing, so that any modification, deletion, or reordering of logs can be identified.

2.2 System Design

The system is based on the concept of data integrity, where any change in stored information must be detectable. Cryptographic hashing is used to generate a unique fingerprint for each log entry. Since each log is linked to the previous one, any modification breaks the chain, making tampering evident. This approach ensures that the logs remain trustworthy and secure.

The logging system is based on a chained structure where each log entry contains important information such as an index, timestamp, event type, description, and a hash value. Each log is connected to the previous one using a "previous hash" field, forming a secure chain.

A cryptographic hash function (SHA-256) is used to generate a unique hash for each log entry. This hash depends on the contents of the log as well as the hash of the previous log. Because of this, even a small change in any log will affect all subsequent logs, making tampering easy to detect.

This approach is similar to how blockchain systems maintain integrity, where data is linked and secured using hashing.

2.3 Implementation

The system was implemented using Python. A class-based approach was used to represent each log entry and manage the collection of logs. Users can dynamically add new log entries through input, making the system interactive.

To verify integrity, the system recalculates the hash of each log and compares it with the stored value. It also checks the sequence of logs and the linkage between them. If any mismatch is found, the system reports the exact log where tampering has occurred.

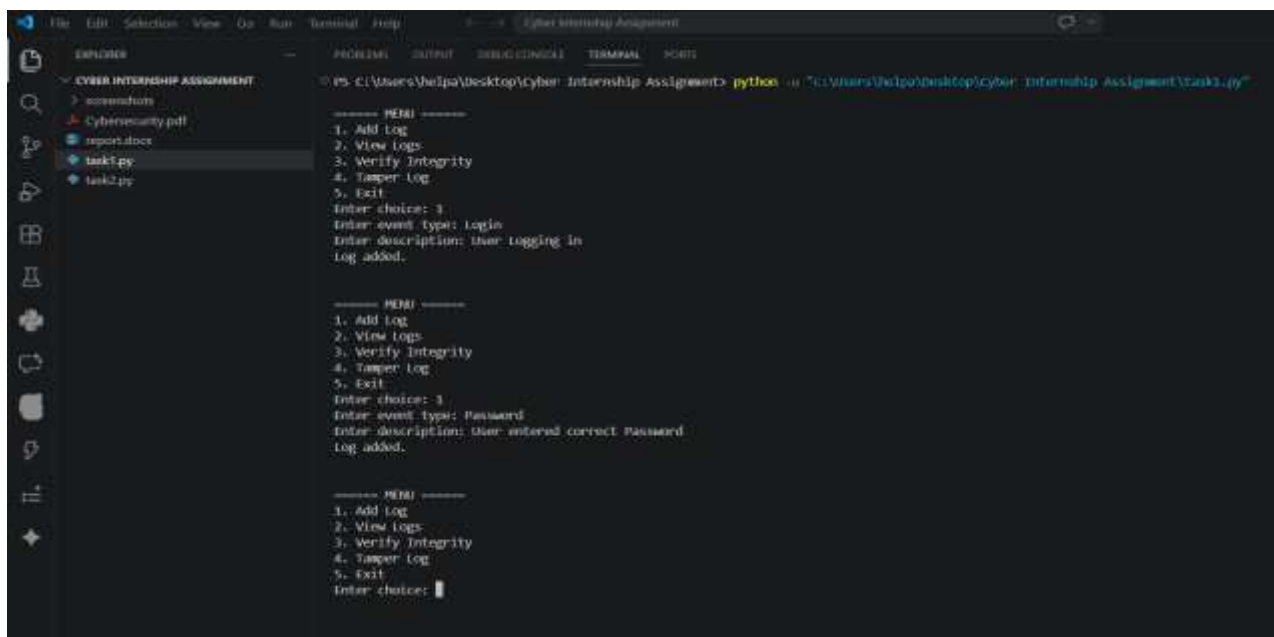
Additionally, a dynamic tampering feature was included to simulate real-world attacks. This allows modification of different fields such as event type, description, timestamp, or index, and helps demonstrate how the system detects such changes.

2.4 Results and Output

The system was tested by adding multiple log entries and verifying their integrity. Initially, all logs were validated successfully. After manually modifying a log entry, the system was able to detect the tampering and identify the affected log along with the altered fields.

The following screenshots demonstrate the working of the system:

(Figure 1: Adding Log Entries)



```
File Edit Selection View Go Run Terminal Help Cyber Internship Assignment
EXPLORES PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
CYBER INTERNSHIP ASSIGNMENT
  screenshots
  Cybersecurity.pdf
  report.docx
  task1.py
  task2.py
PS C:\Users\hulpa\Desktop\Cyber Internship Assignment> python -i "C:\Users\hulpa\Desktop\Cyber Internship Assignment\Task1.py"

----- MENU -----
1. Add Log
2. View Logs
3. Verify Integrity
4. Tamper Log
5. Exit
Enter choice: 1
Enter event type: Login
Enter description: user logging in
Log added.

----- MENU -----
1. Add Log
2. View Logs
3. Verify Integrity
4. Tamper Log
5. Exit
Enter choice: 1
Enter event type: Password
Enter description: user entered correct Password
Log added.

----- MENU -----
1. Add Log
2. View Logs
3. Verify Integrity
4. Tamper Log
5. Exit
Enter choice: 1
```

(Figure 2: Viewing Logs)

```
===== MENU =====
1. Add Log
2. View Logs
3. Verify Integrity
4. Tamper Log
5. Exit
Enter choice: 2

===== LOGS =====

-----
Index: 0
Time: 2026-04-24 19:08:54.098636
Event: Login
Description: User Logging in
Hash: efc9c7a13724efc ...
Prev Hash: 0 ...

-----
Index: 1
Time: 2026-04-24 19:09:50.782759
Event: Password
Description: User entered correct Password
Hash: 3ca29a906f35cf2 ...
Prev Hash: efc9c7a13724efc ...
```

(Figure 3: Integrity Check - No Tampering)

```
===== MENU =====
1. Add Log
2. View Logs
3. Verify Integrity
4. Tamper Log
5. Exit
Enter choice: 3

Checking integrity...

✅ All logs are secure
```

(Figure 4: Before Tampering)

```
Enter log index to tamper: 1

What do you want to change?
1. Event Type
2. Description
3. Timestamp
4. Index (reordering test)
Choice: 2
Before: User entered correct Password
New description: user denied login
After: user denied login

Tampering completed.
```

(Figure 5: After Tampering)

```
Log 0
Time: 1776965252.2600698
Event: User login
Hash: 2172bb602ecc531de2577bc7ea69a5d62fd881b51b155e2267b0b4e022a76982
Previous Hash: 0

Log 1
Time: 1776965252.2606719
Event: File opened
Hash: 24133b247df785a74900d804a0589fe6f7197956da40fef0ae69c3950138c909
Previous Hash: 2172bb602ecc531de2577bc7ea69a5d62fd881b51b155e2267b0b4e022a76982

Log 2
Time: 1776965252.2606804
Event: Payment made
Hash: 0baf013477eea1c6a83d2bb7f277f400a69dd6f38b243e87377cef0394e2908a
Previous Hash: 24133b247df785a74900d804a0589fe6f7197956da40fef0ae69c3950138c909

Before Tampering: ☒ All logs are secure

After Tampering: ☐ Log 1 has been tampered!
PS C:\Users\helpa\Desktop\Cyber Internship Assignment>
```

(Figure 6: Tampering Detection)

```
===== MENU =====  
1. Add Log  
2. View Logs  
3. Verify Integrity  
4. Tamper Log  
5. Exit  
Enter choice: 3  
  
Checking integrity...  
  
✗ Log 1 modified (data tampered)
```


3. Implementation Details

This task was implemented using Python due to its simplicity and flexibility. The tamper-evident logging system was developed using a class-based structure, where each log entry stores its data along with a cryptographic hash. The hashlib library was used to generate SHA-256 hashes, ensuring that each log is securely linked to the previous one.

The implementation focuses on clarity and correctness, making it easy to understand how core security concepts are applied in practice.

4. Why Python Was Chosen?

Python was selected for this assignment because of its readability and ease of use, which makes it suitable for implementing security concepts in a clear and understandable way. It provides built-in libraries such as hashlib that simplify the implementation of cryptographic functions.

Additionally, Python allows quick development and testing of ideas, which is helpful when building systems like logging mechanisms and controlled execution environments. Its simple syntax makes it easier to focus on logic rather than complex language structures.

Overall, Python was an appropriate choice for demonstrating the required concepts effectively.

5. Results & Observations

The implemented system was tested with multiple scenarios to evaluate their effectiveness. In Task, the logging system successfully detected any modifications made to log entries, including changes in data, order, and structure. The system clearly identified the affected log and the type of tampering.

Overall, the system performed as expected and demonstrated the importance of implementing security mechanisms in software systems.

6. Conclusion

This assignment provided practical experience in implementing core cybersecurity concepts. The tamper-evident logging system demonstrated how data integrity can be maintained using cryptographic hashing.

Through this implementation, it became clear that even simple systems can be designed to detect and prevent potential security risks. The assignment also emphasized the importance of writing secure and reliable code.

This assignment was completed through self-learning using various online resources such as documentation, tutorials, and cybersecurity articles. It gave me the opportunity to explore new concepts and apply them in a practical way. While working on this, I was able to develop a clearer understanding of important topics like data integrity, hashing etc. This assignment also reflects my ability to learn independently and implement concepts effectively. I am motivated to keep improving my skills and continue learning more in the field of cybersecurity and software development.

Overall, this work helped in understanding how theoretical security concepts can be applied in real-world scenarios.

7. References

1. Python hashlib documentation

<https://docs.python.org/3/library/hashlib.html>

2. Python official documentation

<https://docs.python.org/3/>

3. GeeksforGeeks – SHA-256 Hashing

<https://www.geeksforgeeks.org/sha-256-hash-in-python/>

4. OWASP – Secure Coding Practices

<https://owasp.org/www-project-secure-coding-practices/>

5. Python eval() function explanation

<https://www.programiz.com/python-programming/methods/built-in/eval>

6. Basic Cybersecurity Concepts (IBM)

<https://www.ibm.com/topics/cybersecurity>